

2025년도

학사학위논문

쿠버네티스 기반 스프링 클라우드
마이크로서비스 아키텍처의 구현

Implementation of a Microservices
Architecture Based on Kubernetes and
Spring Cloud

2025년 05월 27일

순천향대학교 공과대학
컴퓨터공학과

유 동 균

쿠버네티스 기반 스프링 클라우드
마이크로서비스 아키텍처의 구현

Implementation of a Microservices
Architecture Based on Kubernetes and
Spring Cloud

지도교수 이 상 정

이 논문을 공학사학위 논문으로 제출함

2025년 05월 27일

순천향대학교 공과대학

컴퓨터 공학과

유 동 균

유 동 균의 공학사학위논문을 인준함

2025년 05월 27일

심 사 위 원 남 윤 영 인

심 사 위 원 이 상 정 인

순천향대학교 공과대학

컴퓨터 공학과

초록

현대 소프트웨어는 사용자 요구의 다양화로 인해 지속적인 기능 추가와 구조의 복잡성에 직면한다. 이러한 문제점은 시스템을 무겁게 만들며 그로 인해 확장성, 가용성과 같은 핵심 기능을 저해할 수 있다. 특히 단일 배포 단위로 구성된 모놀리식 아키텍처는 하나의 장애나 성능 저하가 전체 시스템에 영향을 미치는 구조적 한계를 지닌다. 이러한 문제를 해결하기 위해 변화에 유연하게 대응할 수 있고 안정적이며 확장 가능한 시스템 아키텍처로의 전환이 요구된다.

본 논문은 문제점의 대안으로 Kubernetes 기반의 Spring Cloud를 활용하여 클라우드 환경에 걸맞은 마이크로서비스 아키텍처를 설계하며 구현한다. 제안된 시스템은 서비스 디스커버리와 API 게이트웨이 등의 구성 요소가 통합된 시스템을 구현하고, 오케스트레이션 기능을 통해 오토 스케일링 및 로드 밸런싱 성능을 향상할 수 있도록 설계한다.

주요어: 마이크로서비스, 오토스케일링, Kubernetes, Spring Cloud

ABSTRACT

Modern software systems are increasingly confronted with continuous feature expansion and architectural complexity due to the diversification of user requirements. These issues often result in heavier systems, which negatively affect essential attributes such as scalability and availability. Monolithic architecture, where all functionalities are deployed as a single unit, suffer from structural limitations, as a single failure or performance bottleneck can impact the entire system. To overcome these limitations, a shift toward a flexible, reliable, and scalable system architecture is required.

This study presents an alternative approach by designing and implementing a microservices architecture based on Kubernetes and Spring Cloud, suitable for cloud-native environments. The proposed architecture incorporates essential components such as service discovery and an API gateway and utilizes orchestration mechanisms to enhance system performance through autoscaling and load balancing.

Key words: Microservices, Autoscaling, Kubernetes, Spring Cloud,

차례

제 1 장 서론	1
제 2 장 이론적 배경	3
2.1 클라우드 네이티브	3
2.2 모놀리식 아키텍처와 마이크로서비스 아키텍처	5
2.3 Spring Cloud.....	7
2.4 Docker	8
2.5 Kubernetes	9
제 3 장 마이크로서비스 아키텍처의 설계 및 구현	10
3.1 마이크로서비스	10
3.1.1 Spring Cloud 구성	10
3.1.2 서브 시스템	13
3.2 서비스 기능	17
3.3 Kubernetes 의 오케스트레이션	21
3.4 오토스케일링	23
3.4.1 HPA	23
3.4.2 KEDA.....	25
제 4 장 서비스 적용 및 성능 평가.....	26
4.1 개발 인프라	26
4.2 시스템 평가 및 분석	28
제 5 장 결론	32
참고문헌.....	33
감사의 글	35

그림 차례

[그림 1] Cloud Native 비교	5
[그림 2] Docker 구조.....	8
[그림 3] Kubernetes 구조.....	9
[그림 4] 마이크로서비스 구조.....	10
[그림 5] Eureka 대시보드	11
[그림 6] RabbitMQ 메시지 브로커 AMQP 구조	14
[그림 7] RabbitMQ 관리 대시보드	15
[그림 8] 컨테이너 구성	16
[그림 9] 고객 등록 시퀀스 다이어그램	19
[그림 10] 설정 전 Deployment.yaml	21
[그림 11] Kubernetes 클러스터의 배포 구조	22
[그림 12] 오토스케일링을 위한 설정 후 Deployment.yaml.....	24
[그림 13] Keda-object.yaml.....	25
[그림 14] 서버 컴퓨터의 하드웨어 사양	27
[그림 15] 고객 등록 기능 테스트	28
[그림 16] 간단한 이벤트 메시지 기능 테스트	29
[그림 17] HPA와 KEDA 초기 상태..	29
[그림 18] Apache JMeter GUI..	30
[그림 19] HPA 와 KEDA 스케일링 후 상태..	30
[그림 20] Kubernetes Pod 구성..	31

표 차례

[표 1] Service의 API	18
[표 2] Infrastructure 환경	26

제 1 장 서 론

다양한 기능을 수용하고 지속적으로 확장되는 애플리케이션은 구조가 점차 복잡해지고, 동시에 처리하는 데이터의 요청량 또한 증가하고 있다. 특히 특정 기능에 트래픽이 집중되는 경우, 모놀리식 구조는 단일 지점의 부하가 전체 시스템 장애로 이어질 가능성이 크다.

본 논문에서는 이러한 문제 상황을 설명하기 위해 가상의 기업인 A사를 설정하고 이를 사례 시나리오로 활용한다. A사는 최근 새로운 서비스 체도를 도입하면서 기존 고객 관리 애플리케이션에 등록되는 인원이 급격하게 증가하였다. 이로 인해 고객 데이터를 저장하고 관리하는 애플리케이션의 요구사항이 확대되었으며, 시스템 부하와 서비스 간의 연동 구조도 복잡해지기 시작했다. 결과적으로 다양한 기술적 문제와 운영상의 비효율이 동시에 발생하고 있는 상황이다. 이에 따라 본 논문에서는 요구사항인 확장성, 가용성 등을 확보하기 클라우드 네이티브 개념을 도입한다.

A사의 기존 모놀리식 아키텍처는 모든 기능이 하나의 배포 단위로 통합되어 있어, 특정 모듈의 수정이나 변경 시 반영이 어렵다. 이 한계를 해결하고자 클라우드 네이티브의 개념 중 하나인 마이크로서비스 아키텍처로 전환을 추진해 기능 단위의 독립적인 개발을 구성한다.

본 논문에서는 Spring Cloud 기반의 프레임워크와 컨테이너 기술을 활용해 아키텍처를 구성한다. 확장성 측면에서는 스프링 클라우드의 게이트웨이 기능을 통해 트래픽을 유동적으로 분산시키며

서비스 인스턴스를 추가적으로 등록해 유연한 조정이 가능하다. 또한 애플리케이션의 기능을 뒷받침 하기위해 NoSQL로 구성된 비관계형 데이터베이스와 비동거적 처리를 위한 메시지 큐를 사용한다. 가용성 측면에서는 kubernetes의 자동 배포와 부하분산을 통해 서비스 중단 없이 지속적인 운영이 가능하도록 한다. 이와 같은 설계 기반으로, 본 논문은 클라우드 네이티브 기반 마이크로서비스 아키텍처를 구현하고 이를 단일 서버 환경에서 테스트를 진행한다. 개발한 기능과 스케일링에 대한 테스트를 통해 요구사항을 충족하는지를 평가하며, 향후 유사한 프로젝트에 있어 본 논문의 시스템이 기술적 발판을 제공할 수 있을 것으로 기대한다.

본 논문의 구성은 다음과 같다. 2장에서는 클라우드 컴퓨팅과 클라우드 네이티브의 개념을 소개한다. 그리고 모놀리식 아키텍처와 마이크로서비스 아키텍처 간의 구조적 비교 및 관련 기술을 설명한다. 3장에서는 마이크로서비스 아키텍처의 설계 과정과 함께, 컨테이너 배포 전략 및 오케스트레이션의 방안을 포함한 전체 시스템 구현 내용을 기술한다. 4장에서는 제안된 시스템을 테스트하고 평가한다.

제 2 장 이론적 배경

2.1 클라우드 네이티브(Cloud Native)

클라우드 네이티브(Cloud Native)는 애플리케이션을 유연하고 확장할 수 있게 설계, 배포, 운영할 수 있도록 하는 소프트웨어 접근 방식이다. 이는 컨테이너, 마이크로서비스, 서비스 메시, 선언적 API, 불변 인프라 등의 기술 요소를 기반으로 하며, 다양한 클라우드 컴퓨팅 환경에서 자동화된 운영과 안정적인 시스템 관리를 가능하게 한다 [1].

클라우드 컴퓨팅은 원격에서 설정할 수 있는 서버, 스토리지, 네트워크 등의 컴퓨팅 자원을 필요에 따라 비용을 지불하고 사용하는 서비스 모델이다. 또한 클라우드 컴퓨팅은 제공 모델에 따라 IaaS, PaaS, SaaS로 구분되며, 배포 방식에 따라 퍼블릭, 프라이빗, 하이브리드 클라우드로 분류된다.

클라우드 네이티브의 기술 구성 요소는 4가지로 컨테이너, 마이크로서비스, DevOps, CI/CD로 구성된다. 컨테이너는 애플리케이션 실행에 필요한 모든 구성 요소를 하나의 패키지로 묶어 격리된 환경에서 실행할 수 있게 하며, 일관된 배포 환경을 제공한다. 마이크로서비스 아키텍처는 단일 애플리케이션을 독립적으로 배포 가능한 서비스 단위로 분리하여 확장성과 유지보수성을 향상시킨다. DevOps는 개발과 운영 간의 협업을 기반으로 애플리케이션과 서비스를 통합적으로 제공하기 위한 방법론이다.

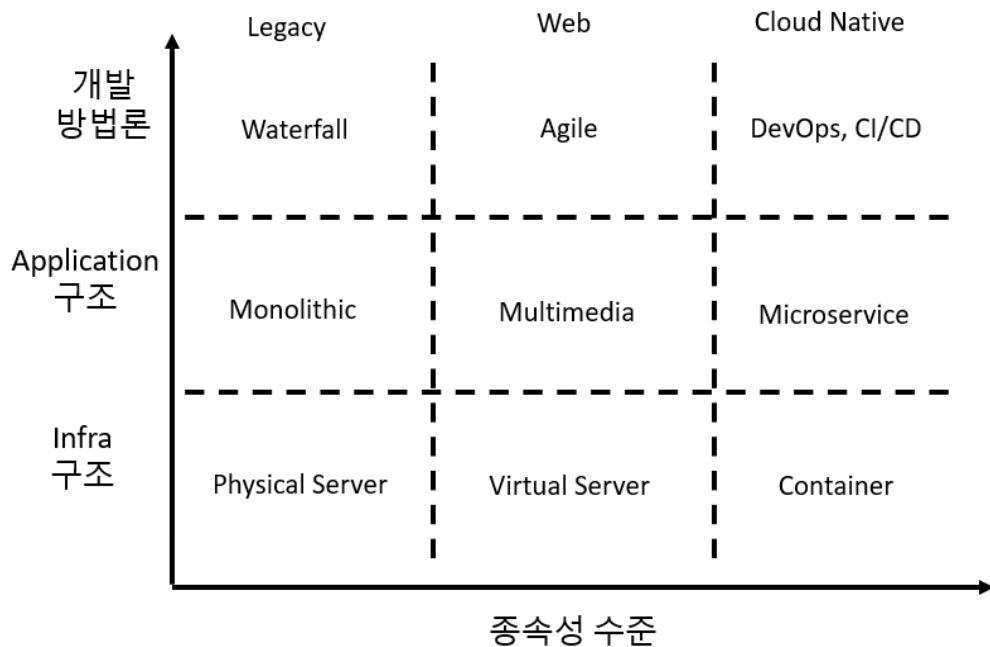
CI/CD(지속적 통합 및 배포) 지속적인 통합과 배포를 자동화하는 파이프라인을 구성하는 DevOps 기반의 개발 프로세스이다.

이러한 기술 요소들은 클라우드 환경에서의 시스템 운영에 있어 중요한 자동화, 복원력, 확장성 등 핵심 특성을 실현한다.

2.2 모듈리식 아키텍처와 마이크로서비스 아키텍처의

모놀리식 아키텍처(Monolithic Architecture)는 다양한 기능의 서비스가 하나의 통합된 코드 베이스의 형태로 동작하는 구조를 의미한다. 내부 구성 요소 간의 높은 결합도와 의존성을 가지며, 하나의 단위로 동작하기 때문에 End-to-End 테스트가 용이하다는 장점을 가진다.

그러나 모놀리식 아키텍처는 A사의 고객 관리 애플리케이션과 같이 시스템의 규모가 커질 경우, 빌드와 배포에 소요되는 시간이 길어지고, 하나의 모듈 변경이 전체 시스템의 재실행을 유발하는 구조적 한계를 지닌다.



[그림 1] Cloud Native 비교 (출처: [참고 2])

마이크로서비스 아키텍처(Microservice Architecture)는 애플리케이션을 기능 단위로 분리된 서비스들로 구성하며, 각 서비스가 독립적인 배포와 확장이 가능한 아키텍처 방법이다. 모놀리식 아키텍처에 비해, 마이크로서비스 아키텍처는 서비스 간의 결합도가 낮아 변경 및 확장에 용이하고, 기능 단위로 서비스가 세분화 되어있기 때문에 하나의 서비스 장애가 시스템에 미치는 영향을 최소화할 수 있는 장점을 가진다.

A사의 애플리케이션이 지향하는 지속적인 기능 확장과 트래픽 변화에 따른 안정적인 서비스 배포 및 운영을 달성하기 위해, 마이크로서비스 아키텍처는 적절한 전환 대안이다.

2.3 Spring Cloud

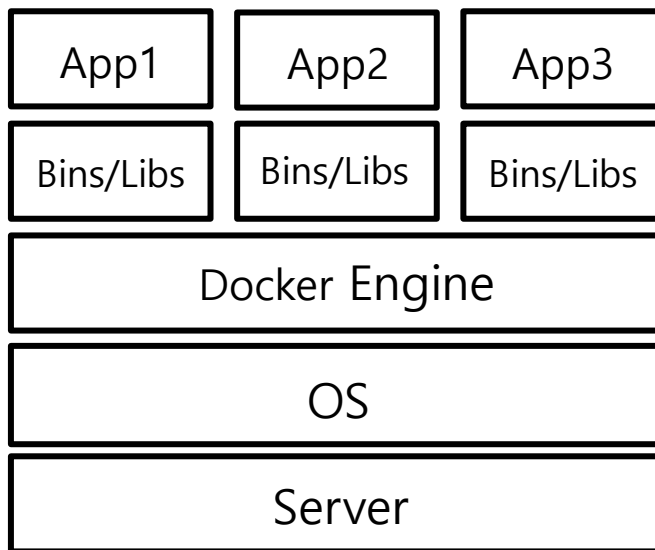
Spring Cloud는 마이크로서비스 아키텍처 기반 시스템 개발을 지원하기 위해 다양한 기능을 제공하는 프레임워크이다. 초기에 Netflix OSS를 바탕으로 시작되었으며, 서비스 디스커버리, 라우팅, 부하 분산, 회복 탄력성 등의 기능이 통합되어 사용할 수 있다. 현재 Spring Cloud는 Eureka, Spring Cloud Config, Spring Cloud Gateway, Actuator 등이 포함된다[3].

Spring Cloud는 각 마이크로서비스 간의 독립성과 확장성을 보장하며, 각 서비스는 Spring Boot 기반으로 구현된다. 구성 요소별 핵심 기능을 통해 시스템의 개발과 운영을 보다 체계적으로 접근할 수 있다. Eureka는 서비스 디스커버리 기능을 제공하여 각 서비스를 동적으로 등록하고 탐색할 수 있도록 한다. Gateway는 API 요청을 수신하여 마이크로서비스로 라우팅하고 디스커버리와 연동을 통해 요청을 분산시킨다. Config는 마이크로서비스의 설정 정보를 중앙 관리하며, 변경 사항을 자동으로 반영함으로써 지속적인 유지보수를 용이하게 한다.

2.4 Docker

컨테이너(Container)는 호스트 운영체제 위에서 애플리케이션과 그 실행 환경을 하나의 패키지로 묶어 이미지화하여 실행하는 기술이다. 운영체제 커널을 공유하면서도 애플리케이션 간의 실행 환경을 분리할 수 있어, 빠르고 가볍게 동작하는 장점을 가진다.

Docker는 그중에서도 컨테이너 기술을 사용하는 애플리케이션의 실행을 위해 설계된 소프트웨어 컨테이너 플랫폼으로, 컨테이너의 실행과 관리를 담당하는 Docker Daemon과 이를 구동하는 Docker Engine으로 구성된다[4].

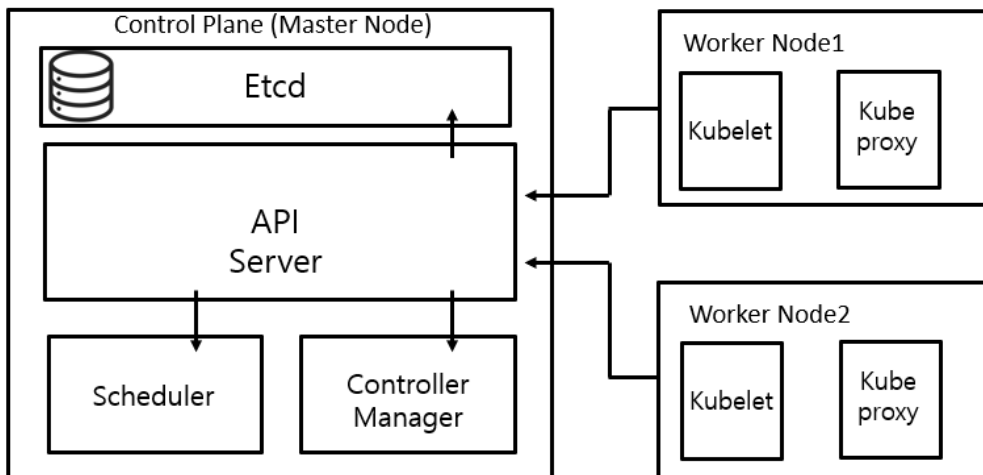


[그림 2] Docker 구조

2.6 kubernetes

Kubernetes는 컨테이너의 상태를 프로비저닝하고 자동으로 배포, 확장 및 조율하는 오케스트레이션 플랫폼이다. 이는 컨테이너를 논리적인 단위인 파드(pod)로 그룹화하여 클러스터 내에서 스케줄링하고 Control Plane과 다수의 Worker Node로 나뉘어 구성된다. Control Plane은 API Server, Scheduler, Controller Manager, etcd 등을 통해 클러스터 상태를 관리한다. Worker Node는 Kubelet, Kube-proxy를 포함하며, 파드를 생성하고 컨테이너를 실행한다.

Kubernetes는 선언적 구성 방식과 상태 관리를 통해 클러스터를 자동으로 조정한다. 로드밸런싱 기능을 제공해 파드에 대해 트래픽을 분산시켜 가용성을 보장한다. 또한, HPA (Horizontal Pod Autosca)와 KEDA(Kubernetes-based Event Driven Autoscaling)를 통해 애플리케이션의 리소스 사용량과 이벤트 기반 워크로드를 감지하여 파드의 수를 조정하고 성능을 유지할 수 있다[5].



[그림 3] Kubernetes 구조

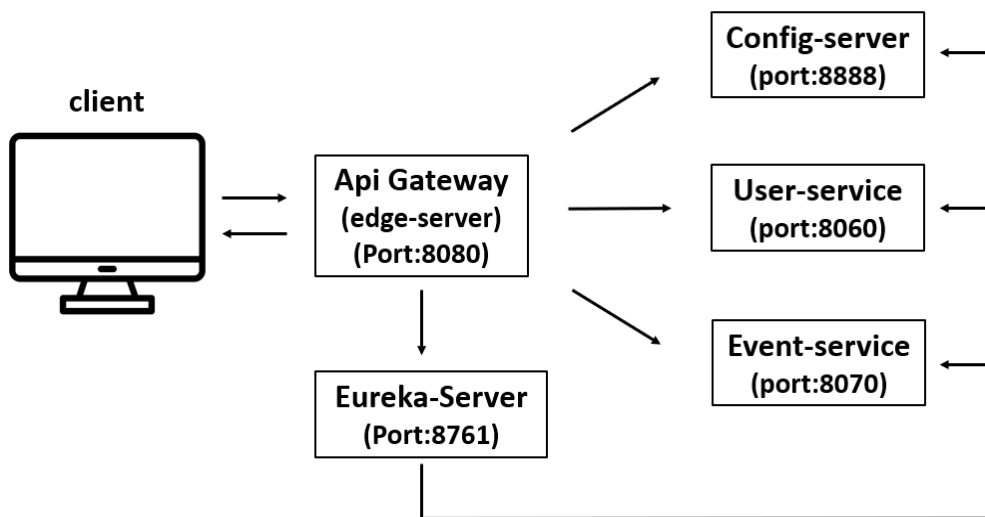
제 3 장 마이크로서비스 아키텍처의 설계 및 구현

3.1 마이크로서비스

3.1.1 Spring Cloud 구성

본 논문은 앞서 설명한 가상 기업 A사의 고객 관리 애플리케이션 개발 및 확장 요구를 위한 솔루션을 제안하며, 이를 기반으로 최소한의 핵심 기능을 구현한다. A사는 고객 관리의 안정성을 확보함과 동시에, 향후 기능 확장을 고려한 구조 개편과 이벤트 기반 비동기 확장성을 요구하였다.

전체 시스템은 Spring Cloud 프레임워크를 기반으로 마이크로서비스를 구현한다. 각 서비스는 독립적으로 설계되어 유연한 운영이 가능하도록 구성되었다. [그림 4]에서 알 수 있듯이 총 5개의 마이크로서비스로 구성되며, 데이터베이스와 메시지 큐도 함께 사용한다.



[그림 4] 마이크로서비스의 구조

Edge Server는 사용자 요청을 API Gateway를 통해 수신하며, 외부로부터 들어온 요청을 내부 마이크로서비스로 전달하는 진입점으로 작동한다. 요청된 URI에 따라 알맞은 서비스를 판별하고, 해당 요청을 적절한 서비스로 라우팅함으로써 트래픽을 효과적으로 분산시킨다. 서비스가 확장되고 배포 진행 시 다수의 인스턴스가 실행되어도 Edge Server의 Gateway는 이를 자동으로 인식하고 적절히 분산시킨다.

Eureka Server는 각 마이크로서비스가 동적으로 등록되고 탐색될 수 있도록 지원하는 서비스 Discovery Server이다. 모든 마이크로서비스는 구동 시 Eureka에 등록되며, Eureka는 등록된 서비스 인스턴스들을 관리하고 모니터링한다. 따라서 서비스들은 IP, 포트 없이 서로를 동적으로 탐색할 수 있다. [그림 5]와 같이 Eureka 대시보드를 통해 등록된 서비스와 각 인스턴스의 상태를 직관적으로 체크할 수 있다.



[HOME](#) [LAST 1000 SINCE STARTUP](#)

System Status

Environmenttest

Data centerdefault

Current time2025-05-18T07:06:44 +0000

Uptime4 days 21:46

Lease expiration enabledtrue

Renews threshold8

Renews (last min)16

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CONFIG-SERVER	n/a (1)	(1)	UP (1) - b71710ef6f0c:config-server:8888
EDGE-SERVER	n/a (1)	(1)	UP (1) - d12bb282b658:edge-server:8080
EVENT-SERVICE	n/a (1)	(1)	UP (1) - 5fac17c70ac3:event-service:8070
USER-SERVICE	n/a (1)	(1)	UP (1) - 26ebb6f6dad6:user-service:8060

General Info

Name	Value
total-avail-memory	224mb
num-of-cpus	32
current-memory-usage	113mb (50%)

[그림 5] Eureka 대시보드

서비스의 상태의 안정성을 보장하기 위해, 각 마이크로서비스는 Spring Boot Actuator를 통해 헬스 체크 엔드포인트(/actuator/health)를 제공한다. Eureka는 이를 주기적으로 호출하여 인스턴스의 정상 여부를 판단한다.

이후의 Kubernetes 환경에서 서비스가 확장되어 여러 파드로 실행될 경우, 각 인스턴스는 Eureka 에 자동 등록되며, API Gateway 는 이를 기반으로 요청을 적절히 분산시킨다.

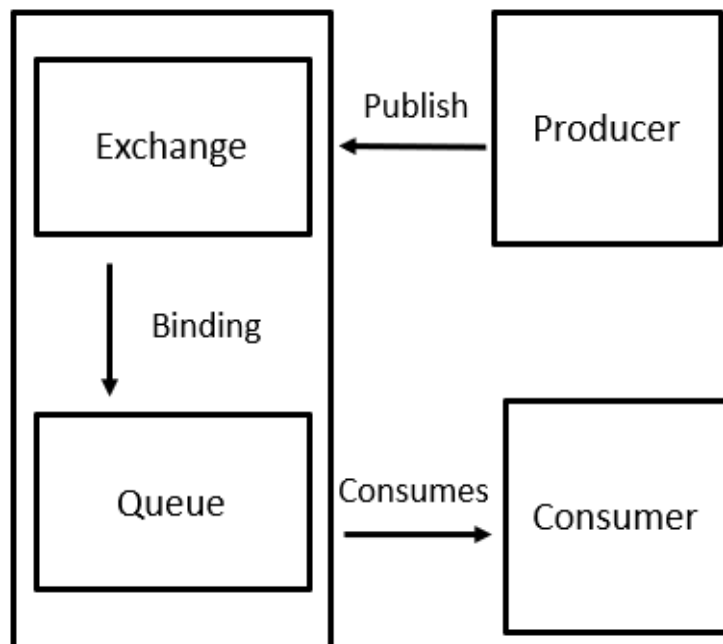
Config Server 는 마이크로서비스의 설정 정보를 중앙에서 일괄적으로 관리하는 구성 서버 역할을 수행한다. 각 마이크로서비스는 구동 시 Config Server 를 통해 설정(Application.yml)을 로드한다. 이는 외부 Git 저장소에 저장되고 관리되도록 구성된다. 각 설정 파일은 YAML 과 Properties 형식으로 작성되며 Dev, Prod 같은 Profile 로 구분되어 적용할 수 있다. 이를 통해 설정이 코드와 분리되어 높은 관리 효율성을 보장한다.

3.1.2 서버 시스템

본 논문에서 데이터베이스는 MongoDB를 사용한다. A사 애플리케이션의 요구사항은 고객 정보 외에 다양한 형태의 데이터가 추가될 수 있어, 정형화된 스키마보다 유연한 데이터 모델이 요구된다. MongoDB는 스키마를 갖지 않는 문서 지향 NoSQL 데이터베이스로, 이러한 요구에 적합하다.

또한 Replica Set 복제 기능을 통해 데이터가 증가에도 수평 확장이 가능하고 가용성 확보에 유리하다. 이러한 특성은 클라우드 네이티브 환경과 높은 호환성을 제공하며, 향후 시스템의 확장에 효과적인 기반을 마련해준다.

메시지 큐는 RabbitMQ를 사용한다. RabbitMQ는 AMQP(Advanced Message Queuing Protocol) 기반의 메시지 브로커로, 메시지 라우팅, 큐잉 등의 기능이 가능하다. RabbitMQ는 생산자(Producer)가 전송한 메시지를 큐에 저장한 후, 소비자(Consumer)가 비동기적으로 구독하여 처리하는 방식으로 동작한다[6].



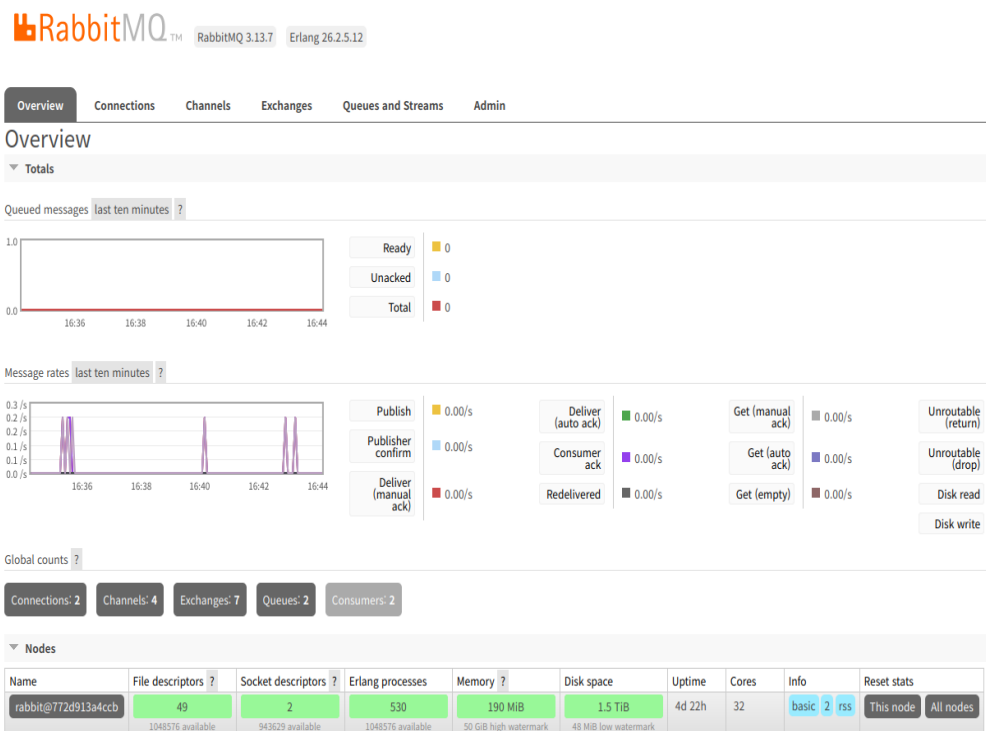
[그림 6] RabbitMQ 메시지 브로커 AMQP

RabbitMQ는 고객의 관리와 주요 이벤트 기능을 비동기적으로 처리하기 위해 필수적이며, Spring Cloud Bus의 설정 메시지 전송 수단으로도 사용된다.

Spring Cloud Bus는 Config Server에서 설정 정보가 변경되었을 때 마이크로서비스에 브로드캐스트하여 실시간으로 반영되도록 지원하는 기능이다.

RabbitMQ를 활용함으로써, 설정 변경을 동적으로 반영하고 서비스 재시작 없이도 자동 반영되는 환경을 구축한다.

따라서 본 RabbitMQ는 비동기 이벤트 중심 아키텍처의 핵심 수단으로 활용될 뿐만 아니라, 설정 정보의 동적 반영을 위한 인프라로 기능한다.



[그림 7] RabbitMQ 관리 대시보드

모든 마이크로서비스는 Docker 컨테이너로 구성되어 있으며, Docker Compose.yml 설정 파일을 통해 이를 통합적으로 관리한다. 각 마이크로서비스는 내부적으로 고유한 포트 번호를 사용해 구분되며, [그림 8]에서 확인할 수 있다.

서비스를 정상적으로 기동하기 위해서는 지켜야 할 실행 순서가 존재한다. 먼저, 서비스 디스커버리를 위해 Eureka Server를 먼저 실행시켜 다른 서비스를 등록시킨다. 이후 순서로 Config Server를 구동하여 각 설정을 중앙 관리하며, 그다음 User, Event Service들을 실행시킨다. 마지막으로 Edge Server를 실행시켜 API Gateway를 활성화하고 시스템을 통합 실행시킨다.

```
hadoop@halla:~/donggyun/CRMCloud$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
569ce779099c	48d9cfaaf390	"/metrics-server --c."	14 hours ago	Up 14 hours	
k8s_metrics-server_metrics-server-596474b58-lcc9b_kube-system_6d444480-088d-4c96-a85e-539acf766ceb_0					
e76345319ba0	registry.k8s.io/pause:3.6	"/pause"	14 hours ago	Up 14 hours	
k8s_POD_metrics-server-596474b58-lcc9b_kube-system_6d444480-088d-4c96-a85e-539acf766ceb_0					
987691aba136	crmcloud_edge-server	"java -jar app.jar"	16 hours ago	Up 16 hours	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp
crmcloud_edge-server_1					
34fe89677a43	crmcloud_event-service	"java -jar app.jar"	16 hours ago	Up 16 hours	0.0.0.0:8070->8070/tcp, :::8070->8070/tcp
crmcloud_event-service_1					
d89d7b18f1aa	crmcloud_user-service	"java -jar app.jar"	16 hours ago	Up 16 hours	0.0.0.0:8060->8060/tcp, :::8060->8060/tcp
crmcloud_user-service_1					
ce761920ef7b	crmcloud_config-server	"java -jar app.jar"	16 hours ago	Up 16 hours	0.0.0.0:8888->8888/tcp, :::8888->8888/tcp
crmcloud_config-server_1					
5e38e3d58ce7	mongo	"docker-entrypoint.s..."	16 hours ago	Up 16 hours	0.0.0.0:27017->27017/tcp, :::27017->27017/t
crmcloud_mongodb_1					
a48148193904	crmcloud_eureka-server	"java -jar app.jar"	16 hours ago	Up 16 hours	0.0.0.0:8761->8761/tcp, :::8761->8761/tcp
crmcloud_eureka-server_1					
8d8337e4876b	rabbitmq:3-management	"docker-entrypoint.s..."	16 hours ago	Up 16 hours	4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, /tcp
crmcloud_rabbitmq_1					

[그림 8] 컨테이너 구성

3.2 서비스 기능

서비스 기능으로 고객 정보 관리 CRUD 기능과 이벤트 메시지 처리의 기능을 중심으로 구성되었으며, 주요 로직은 User-Service와 Event-Service를 통해 설계되었다. 각 서비스는 마이크로서비스 간 결합도를 최소화하고 유기적으로 통합되어 설계된다.

User Service는 고객의 등록, 조회, 수정, 삭제 기능을 담당하는 핵심 도메인 서비스이다. 고객 등록 시 클라이언트로부터 입력받은 이름, 이메일, 비밀번호 등의 정보는 유효성 검사를 거쳐 데이터베이스에 저장되며, 비밀번호는 Spring Security에서 제공되는 BCryptPasswordEncoder를 사용해 단방향 해시 방식으로 암호화되어 처리된다. 또한, 각 고객에게는 고유 식별자ID와 핸드폰 기기번호가 무작위로 부여되어, 데이터의 중복을 방지하고 시스템 내 고유성을 유지한다.

로직은 Spring Boot 기반의 MVC(Model-View-Controller) 패턴을 적용하여 구현한다. 데이터 관련 로직은 Repository에서 MongoRepository를 상속받아 사용되며, 복잡한 쿼리 없이 기본적인 CRUD 기능을 구현할 수 있다. 시스템 전반에서 사용되는 암호화 모듈, 객체 매핑도구(ModelMapper), 메시지 브로커(RabbitTemplate)와 같은 객체들은 Spring의 의존성 주입 DI(Dependency Injection)기반으로 구성되며, 이를 @Configuration 어노테이션을 통해 빈(Beans)으로 등록해 관리하고 사용한다. 이러한 구성 요소들은 Spring 컨테이너에 의해 싱글톤(Singleton) 패턴으로 관리되며, 일관된 객체를 공유함으로써 불필요한 인스턴스 생성 없이 효율적으로 사용할 수 있다.

[표 1] Service의 API

Service	REST API
고객 등록	POST /customers
고객 전체 조회	GET /customers/all
특정 고객 업데이트	POST /customers/{id}
특정 고객 삭제	DELETE /customers/{id}
간단 메시지 전송	POST /messages

조회 기능은 데이터베이스에 저장된 고객 정보를 모두 불러오는 기능으로, REST API 의 GET Method 를 기반으로 구현되었다. 클라이언트는 GET /customers/all 경로를 통해 조회 요청을 전송하며, 전체 고객 정보를 HTTP 응답으로 반환한다. 클라이언트가 시스템에 저장된 고객 현황을 실시간으로 확인할 수 있도록 지원한다.

생성 기능은 새로운 고객 정보를 등록하는 기능으로, POST 메서드를 기반으로 구현되었다. POST /customers 경로를 통해 요청 본문에 고객 정보를 담아 전송하며, 데이터베이스에 저장 후 HTTP 응답을 결과로 반환한다.

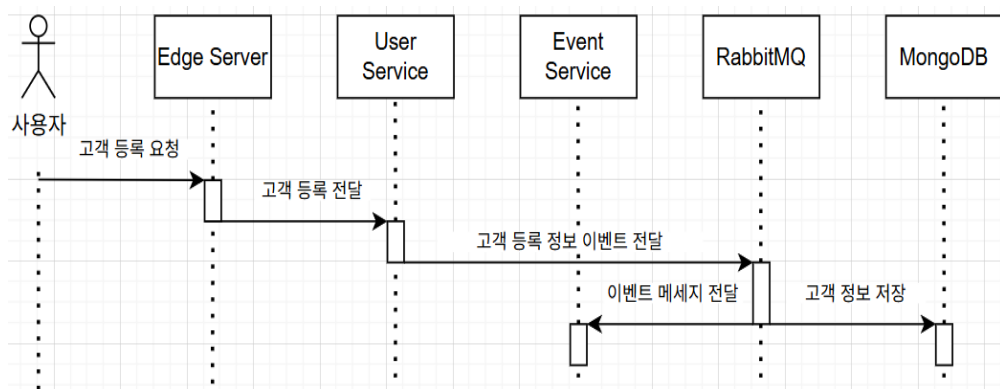
업데이트 기능은 기존 고객의 정보를 수정하는 기능으로, POST메서드를 기반으로 구현되었다. POST /customers/{id} 경로를 통해 수정할 고객의 ID를 전달하고, 변경 정보를 함께 전송한다. 서버는 해당 ID에 해당하는 데이터를 찾아

수정 후 결과를 응답한다.

삭제 기능은 특정 고객의 정보를 제거하는 기능으로, DELETE 메서드를 기반으로 사용했다. DELETE /customers/{id} 형태로 요청을 전송하고 해당 ID에 고객 데이터를 데이터베이스에서 삭제 후 결과를 응답한다.

각 REST API의 메서드는 Spring Framework에서 제공하는 어노테이션을 통해 구현된다. @GetMapping, @PostMapping 등과 같은 어노테이션을 사용하여 URL 경로를 연계해 구성하였고 이러한 방식은 RESTful 인터페이스 설계에 부합하며, API의 명확성과 확장성을 높인다.

이벤트 메시지 기능으로 고객 등록 시 이벤트를 발생해 동기적으로 처리하는 알람 기능과 간단한 메시지를 주고받는 기능을 설계하였다. Event Service로 중심 작동하며, RabbitMQ를 기반으로 비동기 이벤트 처리를 진행한다.



[그림 9] 고객 등록 시퀀스 다이어그램

고객 정보 등록 시 User Service는 저장된 데이터를 기반으로 이벤트 메시지를 생성하고, 이를 RabbitMQ의 큐에 전송함으로써 이벤트를 발생시킨다. 이후 Event Service는 해당 큐를 구독하여 수신하고, 그 내용을 로그로 출력하거나 향후 처리 로직에 활용할 수 있도록 동작한다. 메시지는 JSON 형태로 직렬화되어 전송되며, Spring의 @RabbitListener 어노테이션을 활용해 큐로부터 메시지를 자동으로 수신받고 처리할 수 있도록 구성된다. 간단한 문자열 기반의 메시지를 송수신하는 기능도 구현하여 메시지 큐 기반의 비동기 통신 흐름을 테스트할 수 있는 구조를 마련한다.

이벤트 메시지 기능은 향후 애플리케이션이 확장되어 다양한 비동기 이벤트가 동시에 발생할 경우를 대비한 설계이며, 트래픽 과부하에 대한 대응 가능성을 실험적으로 검증하기 위한 초기 단계로 구현했다. 이 기능은 Kubernetes 환경에서 자동 확장과 로드밸런싱이 결합되어, 이벤트 수신량이나 메시지 처리량에 따라 서비스 인스턴스를 유동적으로 조절할 수 있는 아키텍처로 발전시킬 수 있다. 이를 통해 높은 트래픽 환경에서도 안정적인 이벤트 처리가 가능하며, 클라우드 네이티브 아키텍처의 원칙에 맞게 설계한다.

3.3 kubernetes의 오케스트레이션

앞서 구성했던 마이크로서비스 아키텍처를 자동화된 방식으로 배포하고 운영하기 위해, Kubernetes를 기반의 오케스트레이션 환경을 적용한다. 마이크로서비스의 각 서비스를 Kubernetes의 선언적인 구성 방식에 따라 deployment.yaml과 service.yaml파일로 정의하여 배포한다.

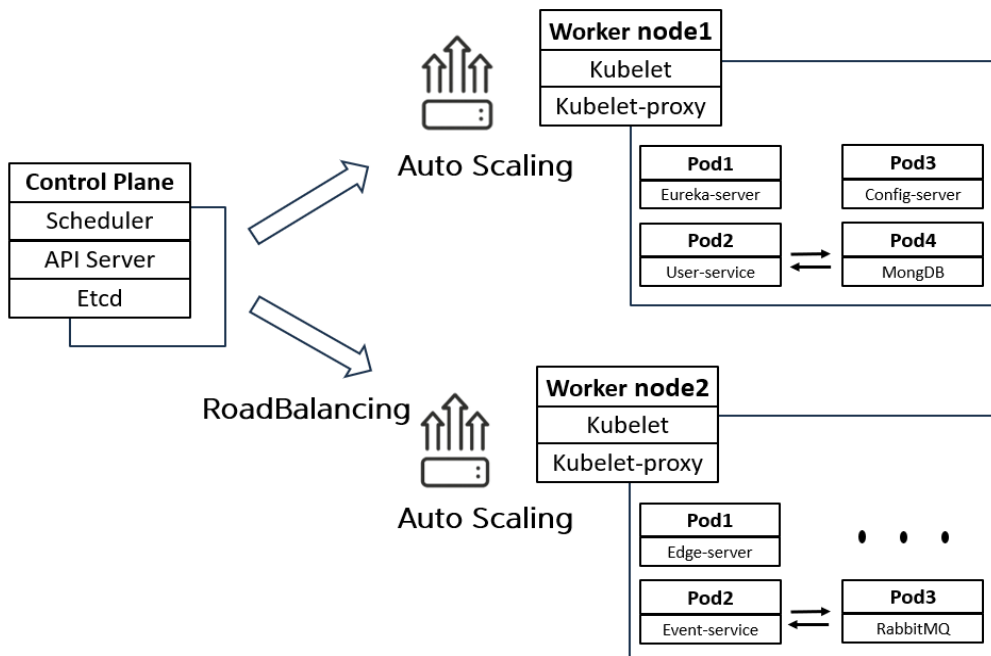
Deployment는 애플리케이션의 상태를 정의하는 역할을 하며, 컨테이너 이미지, Pod의 수 등을 명시함으로써 애플리케이션의 자동 배포, 스케일링, 복구 관리가 가능하게 된다. Replicas를 2로 설정하여 2개의 파드 인스턴스를 생성하며, containers에 컨테이너에 대한 이미지와 포트 등의 설정을 넣을 수 있다.

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: user-service
5  labels:
6    app: user-service
7  spec:
8    replicas: 2
9    selector:
10   matchLabels:
11     app: user-service
12   template:
13     metadata:
14     labels:
15       app: user-service
16     spec:
17       containers:
18       - name: user-service
19         image: crmcloud_user-service:latest
20         imagePullPolicy: Never
21         ports:
22         - containerPort: 8060
```

[그림 10] 설정 전 Deployment.yaml

Service는 Kubernetes 클러스터 내에서 동적으로 생성되고 변경되는 파드에 대해 고정된 네트워크 접근지점을 제공하는 컴포넌트이다. DNS 이름과 가상 IP를 지정함으로써, 네트워크를 추상화하고 Pod 간 통신을 일정하게 유지시킨다.

데이터베이스와 메시지 큐는 Kubernetes에서 구성할 때 앞서 정의한 배포 파일 외에 PV(Persistent Volume)와 PVC(Persistent Volume Claim)를 함께 설정해 사용한다. PV는 관리자가 Storage class를 사용하여 동적으로 프로비저닝한 스토리지 자원이며, PVC는 PV에서 설정한 스토리지를 Pod에서 사용할 수 있도록 요청하고 마운트하기 위한 선언적 요청 리소스이다. 이를 통해 컨테이너와 독립된 스토리지 공간을 선언하고, 데이터의 영속성과 안정성을 확보할 수 있다.



[그림 11] Kubernetes 클러스터의 배포 구조

3.4 오토스케일링

3.4.1 HPA

HPA(Horizontal Pod Autoscaling)은 Kubernetes에서 제공하는 오토스케일링 기능이다. CPU, 메모리와 같은 리소스가 정해진 임계치를 초과할 경우, 자동으로 파드 수를 조절하는 Scale Out 컨트롤러이다. 이를 통해 시스템은 트래픽 증가와 리소스 과부화에 실시간으로 대응할 수 있다[5].

앞서 구성한 기본 Deployment에서는 replicas를 2로 고정된 파드를 설정하여, 단순한 확장만이 가능한 구조였다. 이는 급격한 트래픽 증가와 리소스 과부화에 적합하지 않으며, HPA를 통한 동적 확장 기반의 오토스케일링이 필요하다.

이를 위해 [그림 10] 파일에 resources.requests 항목을 추가하여, CPU와 메모리의 최소 및 최대 자원 사용량을 명시한다. 이는 HPA가 리소스 사용률을 기반으로 스케일링 여부를 판단하는 기준이 된다.

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: user-service
5    labels:
6      app: user-service
7  spec:
8    replicas: 2
9    selector:
10   matchLabels:
11     app: user-service
12   template:
13     metadata:
14       labels:
15         app: user-service
16     spec:
17       containers:
18         - name: user-service
19           image: crmcloud_user-service:latest
20           imagePullPolicy: Never
21           ports:
22             - containerPort: 8060
23           livenessProbe:
24             httpGet:
25               path: /actuator/health
26               port: 8060
27             initialDelaySeconds: 10
28             periodSeconds: 10
29           readinessProbe:
30             httpGet:
31               path: /actuator/health
32               port: 8060
33             initialDelaySeconds: 5
34             periodSeconds: 5
35           resources:
36             requests:
37               cpu: "100m"
38               memory: "128Mi"
39             limits:
40               cpu: "500m"
41               memory: "512Mi"

```

[그림 12] 오토스케일링을 위한

설정 후 Deployment.yaml

또한, [그림 12]와 같이 livenessProbe와 readinessProbe를 설정하여 각각 컨테이너의 상태와 서비스 수신을 확인할 수 있도록 구성했다. 이 두 요소는 파드가 확장된 이후에도 지속적으로 점검해 오토스케일링 기능을 구현한다.

HPA를 적용하기 위해 별도의 hpa.yaml 파일을 정의해야 한다. 해당 파일에는 확장 대상의 Deployment 이름과 최소 및 최대 파드 수, 리소스의 임계치 등이 포함된다.

3.4.2 KEDA

KEDA(Kubernetes-based Event Driven Autoscaling)는 Kubernetes 클러스터에서 이벤트 기반으로 자동확장을 지원하는 오토스케일링 툴이다. 기존 HPA는 CPU와 메모리 같은 리소스 사용률에 기준으로 조절되었지만, KEDA는 메시지 큐, 다양한 이벤트 등을 기준으로 스케일링이 가능하다는 차별점을 가진다[10].

고객 등록 시 발생하는 이벤트와 간단한 메시지 송수신과 같은 RabbitMQ를 사용하는 기능에 있어, 메시지 처리 부하에 따라 서비스 확장을 위해 KEDA를 도입한다. Keda-object.yaml파일을 작성하여 KEDA를 설정하고 사용한다. 해당 파일에는 스케일링의 대상이 되는 서비스 Event Service를 명시하고, 자동 확장의 최소 및 최대 파드 수 ReplicaCount를 설정한다. 사용되는 메시지 큐의 타입은 RabbitMQ로 지정해 오토스케일링이 이뤄지도록 구성한다.

```
1  apiVersion: keda.sh/v1alpha1
2  kind: ScaledObject
3  metadata:
4    name: event-service-scaledobject
5    namespace: default
6  spec:
7    scaleTargetRef:
8      name: event-service
9    minReplicaCount: 1
10   maxReplicaCount: 5
11   triggers:
12     - type: rabbitmq
13       metadata:
14         queueName: customer-queue
15         queueLength: "20"
16         host: "amqp://admin:000001@rabbitmq.default.svc.cluster.local:5672/"
```

[그림 13] keda-object.yaml

제 4 장 서비스 적용 및 성능 평가

4.1 개발 인프라

본 논문은 개발과 테스트의 용이성을 고려하여 Ubuntu 기반의 단일 서버에서 구성한다. Control Plane과 Worker node를 물리적으로 분해하지 않고 하나의 컴퓨터 시스템 내에서 운영해 구성한다. 부가적인 개발 인프라는 [표 2]로 정리한다.

[표 2] Infrastructure 환경

Tech Stack	Version	Etc
Spring Boot	3.2.4	서비스 프레임워크
Spring Cloud	2023.0.1	애플리케이션 프레임워크
MongoDB	7.0	정보 저장 데이터베이스
RabbitMQ	3.12	이벤트 메시지 큐
Maven	Apache Maven 3.6.3	프로젝트 빌드
Kubernetes	Kubeadm / v1.28.15	클러스터 오케스트레이션
Linux	Ubuntu 22.04 LTS	운영체제

또한, 이후 실행한 스케일링의 테스트를 위해 서버 컴퓨터의 시스템 하드웨어 사양을 [그림 14]를 통해 제시한다.

```
hadoop@halla:~/donggyun/CRMCloud$ lscpu
Architecture:          x86_64
  CPU op-mode(s):      32-bit, 64-bit
  Address sizes:        43 bits physical, 48 bits virtual
  Byte Order:           Little Endian
CPU(s):                 32
  On-line CPU(s) list: 0-31
Vendor ID:              AuthenticAMD
  Model name:           AMD Ryzen 9 3950X 16-Core Processor
    CPU family:         23
    Model:              113
    Thread(s) per core: 2
    Core(s) per socket: 16
    Socket(s):          1
    Stepping:           0
    Frequency boost:    enabled
CPU max MHz:            4761.2300
CPU min MHz:            2200.0000
BogoMIPS:               6987.07
```

[그림 14] 서버 컴퓨터의 하드웨어 사양

4-2. 시스템 평가 및 분석

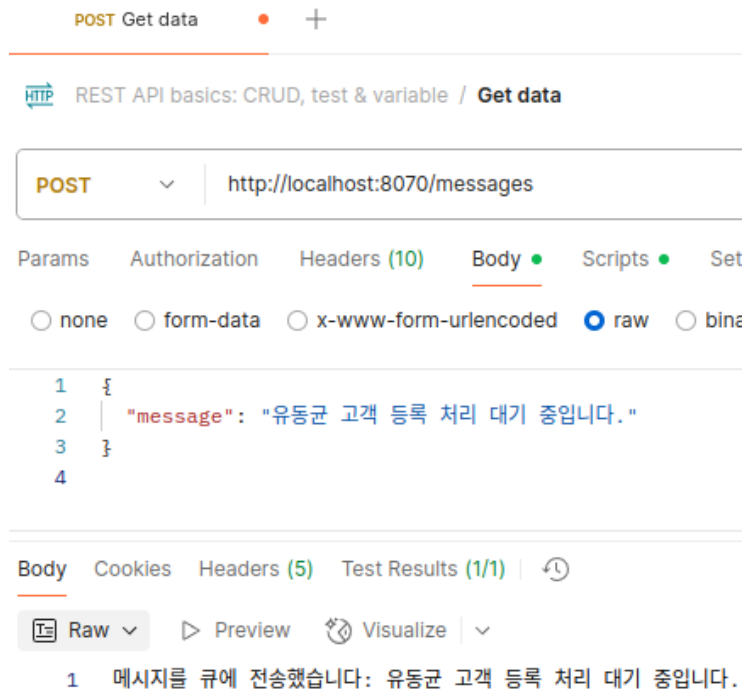
앞서 구현한 마이크로서비스 아키텍처의 기능을 테스트하고 확인한다. [그림 15]를 통해 고객의 등록 기능을 확인할 수 있다.

The screenshot shows a REST client interface with a POST request to `http://localhost:8060/customers`. The 'Body' tab is selected, and the request body is a JSON object. The body is displayed in a syntax-highlighted editor with line numbers 1 through 9. The JSON structure is as follows:

```
1 {
2   "name": "유동균",
3   "email": "a12345@gmail.com",
4   "password": "abc123456",
5   "phone": {
6     "phoneNumber": "01029294949",
7     "deviceSerialNumber": "12bf8a1c-35f4-4eb7-abb5-b1b8382e1cbf"
8   }
9 }
```

[그림 15] 고객 등록 기능 테스트

[그림 16]을 통해 간단한 이벤트 메시지 기능을 확인할 수 있다.



[그림 16] 간단한 이벤트 메시지 기능 테스트

Kubernetes 클러스터 상의 기능들을 각 테스트의 출력을 통해 확인한다.

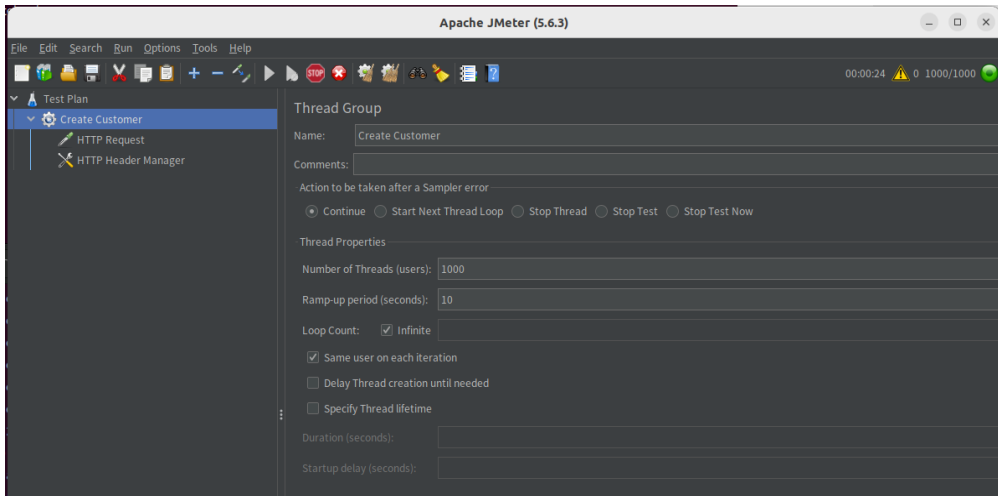
[그림 17]은 초기 HPA 의 사용과 KEDA 의 관리되는 파드의 상황을 확인할 수 있다.

```
hadoop@halla:~/donggyun/CRMCloud$ kubectl get hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
keda-hpa-event-service-scaledobject	Deployment/event-service	0/20 (avg)	1	5	1	4d20h
user-service-hpa	Deployment/user-service	7%/50%	1	5	1	4m48s
user-service-hpa	Deployment/user-service	9%/50%	1	5	1	5m15s
user-service-hpa	Deployment/user-service	6%/50%	1	5	1	5m30s

[그림 17] HPA와 KEDA 초기 상태

HPA, KEDA의 정상적인 작동을 테스트하기 위해 의도적인 트래픽을 유발시켜 확인한다. 애플리케이션의 주요 기능 중 고객의 등록 기능 /customers 경로에서 트래픽을 만들어 진행했다.



[그림 15] Apache JMeter GUI

트래픽 유도는 Apache JMeter 도구를 사용하였으며, 1000명의 가상 사용자가 1초 간격을 통해 고객 등록 요청을 무한 반복하여 수행하도록 테스트 시나리오를 실행하였다.

user-service-hpa	Deployment/user-service	130%/50%	1	5	1	6m05s
user-service-hpa	Deployment/user-service	150%/50%	1	5	5	6m20s
user-service-hpa	Deployment/user-service	213%/50%	1	5	5	6m35s
user-service-hpa	Deployment/user-service	178%/50%	1	5	5	6m50s
keda-hpa-event-service-scaledobject	Deployment/event-service	38/20 (avg)	1	5	5	4d20h

[그림 18] HPA와 KEDA의 스케일링 후 상태

그 결과, HPA는 CPU 사용률이 최대 213%에 도달하여 설정된 목표치인 50%를 초과하였고, 이에 따라 최소 1개에서 최대 5개로 정의된 파드 수 범위 내에서 자동으로 확장되어, 최종적으로 5개의 파드가 활성화된 상태로 스케일링 되었다. 동시에, Event Service에 연결된 큐에 메시지가 지속적으로 유입되어,

큐의 메시지 수가 38건에 도달하였다. 이는 정의된 기준 20건을 초과하였으며, 결과적으로 KEDA는 최대 파드 수인 5개까지 자동 확장되어 분산 처리하는 것을 확인하였다.

[그림 19]를 통해 앞서 작성했던 파일들을 통해 생성한 파드를 확인할 수 있다.

```
hadoop@halla:~/donggyun/CRMLoud$ kubectl get all -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	pod/config-server-6fcc67b7b6-hx7nv	1/1	Running	0	34h
default	pod/edge-server-7c4cf56765-cmtqp	1/1	Running	0	34h
default	pod/eureka-server-56cdfc744d-qgdpr	1/1	Running	0	33h
default	pod/event-service-7ccb88c4c-k5fxx	1/1	Running	0	34h
default	pod/mongodb-657f8488f8-97nfz	1/1	Running	0	6d18h
default	pod/rabbitmq-65d88bd48c-qkzzh	1/1	Running	0	6d18h
default	pod/user-service-7767f55bb7-pt997	1/1	Running	0	33h
keda	pod/keda-admission-webhooks-69dc85dfcc-d2txc	1/1	Running	0	36d
keda	pod/keda-operator-554cb64d9c-6hf4j	1/1	Running	1 (6d18h ago)	36d
keda	pod/keda-operator-metrics-apiserver-58f44c77-2xbsj	1/1	Running	0	36d
kube-system	pod/calico-kube-controllers-5fc7d6cf67-56fwp	1/1	Running	0	6d20h
kube-system	pod/calico-node-8jnvx	1/1	Running	0	6d20h
kube-system	pod/coredns-5dd5756b68-wvqzs	1/1	Running	0	42d
kube-system	pod/coredns-5dd5756b68-x6b9l	1/1	Running	0	42d
kube-system	pod/etcd-halla	1/1	Running	2	42d
kube-system	pod/kube-apiserver-halla	1/1	Running	2	42d
kube-system	pod/kube-controller-manager-halla	1/1	Running	2	42d
kube-system	pod/kube-proxy-r8gh6	1/1	Running	2	42d
kube-system	pod/kube-scheduler-halla	1/1	Running	2	42d
kube-system	pod/metrics-server-596474b58-lcc9b	1/1	Running	0	12h

NAMESPACE	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
default	service/config-server	ClusterIP	10.109.156.195	<none>	8888/TCP	34h
default	service/edge-server	NodePort	10.107.249.225	<none>	8080:32313/TCP	34h
default	service/eureka-server	ClusterIP	10.97.75.152	<none>	8761/TCP	34h
default	service/event-service	ClusterIP	10.105.8.244	<none>	8070/TCP	34h
default	service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	42d
default	service/mongodb	ClusterIP	10.107.23.205	<none>	27017/TCP	6d18h
default	service/rabbitmq	ClusterIP	10.101.5.126	<none>	5672/TCP, 15672/TCP	6d18h
default	service/user-service	ClusterIP	10.100.105.48	<none>	80/TCP	6d18h
keda	service/keda-admission-webhooks	ClusterIP	10.108.144.50	<none>	443/TCP	36d
keda	service/keda-operator	ClusterIP	10.110.214.165	<none>	9666/TCP	36d
keda	service/keda-operator-metrics-apiserver	ClusterIP	10.105.95.62	<none>	443/TCP, 8080/TCP	36d
kube-system	service/kube-dns	ClusterIP	10.96.0.10	<none>	53/UDP, 53/TCP, 9153/TCP	42d
kube-system	service/metrics-server	ClusterIP	10.99.19.244	<none>	443/TCP	33h

[그림 19] Kubernetes Pod 구성

제 5 장 결론

본 논문은 모놀리식 아키텍처로 구성되어 있는 A사의 고객 관리 애플리케이션을 대상으로, 높은 안정성과 확장성을 보장할 수 있는 마이크로서비스 아키텍처로의 전환 방안을 설계하고 구현하였다.

서비스 간의 독립적인 관리를 위해 Spring Cloud기반의 마이크로서비스 아키텍처를 구성하였으며, 이를 통해 A사 애플리케이션의 서비스 단위 확장성과 구조적 효율성을 보완하였다. 또한 RabbitMQ 기반의 이벤트 메시지 구조를 적용함으로써, 다양한 이벤트 처리와 비동기 처리 환경을 마련하였다.

배포와 운영의 자동화를 위해 Kubernetes 환경을 활용하였으며, HPA와 KEDA기반의 오토스케일링 기능을 통해 A사 애플리케이션의 안정성과 가용성 확보에 적합한 시스템을 구현하였다.

본 논문은 이러한 시스템 구성 방안을 통해, 클라우드 네이티브 환경에 걸맞은 확장 가능하고 안정적인 아키텍처 설계하고 구현하였으며, 향후 유사한 프로젝트에 있어 기술적 발판과 설계 방향을 제시하고자 한다.

참 고 문 헌

[1] Cloud Native Computing Foundation, "CNCF Cloud Native Definition v1.1", 2024.02

<https://github.com/cncf/toc/blob/main/DEFINITION.md#%ED%95%9C%EA%B5%AD%EC%96%B4>

[2] 오케스트로, "클라우드 사전 클라우드 네이티브"

https://www.okestro.com/post_archive/what-is-cloud-native/

[3] Spring Cloud Project, Spring 공식 문서

<https://spring.io/projects/spring-cloud>

[4] D. N.kim, "도커(Docker) 입문편: 컨테이너 기초부터 서버 배포까지",
2020.03

<https://www.44bits.io/ko/post/easy-deploy-with-docker>

[5] Kubernetes 공식 문서 Kubernetes.io,

<https://kubernetes.io/ko/docs/home/>

[6] RabbitMQ 공식 문서,

<https://www.rabbitmq.com/docs>

[7] adjh54, "Spring Cloud 프로젝트 구성 및 설명", 2023.06

<https://adjh54.tistory.com/207>

[8] jaehere, "초보를 위한 쿠버네티스 시작 실습", 2025.02

<https://velog.io/@jaehere/초보를-위한-쿠버네티스-쿠버네티스-시작-실습>

[9] mrcocoball2, "Spring Cloud + Kubernetes 기본 개념 및 활용", 2024.01

<https://velog.io/@mrcocoball2/Spring-Cloud-Spring-Cloud-Kubernetes-기본-개념-및-활용>

[10] Steven Cheng, "KEDA (Kubernetes Event-driven Autoscaling) 적용기",
2023.06

<https://medium.com/@scheng1028/keda-kubernetes-event-driven-autoscaling-적용기-ec775c0715ff>

[11] Thomas Vitale, Cloud Native Spring in Action,

<https://github.com/ThomasVitale/cloud-native-spring-in-action>

감사의 글

대학교 생활에 있어서 항상 뒤에서 도와주시고 응원해 주셨으며 바쁠 때마다 집에서 학교까지 매번 차로 데려다주신 부모님께 먼저 감사 인사를 드립니다.

2023년 겨울방학에 CS 연구실에 들어가 생활하였고 그 후로 겪은 모든 일에 대해서 매번 서로 들어주고 챙겨주고 도와준 CS 연구실 부원들 모두 감사합니다. 먼저 졸업한 전 랩장인 윤준식 학우에게는 도움을 많이 받았으며 학교에 대해서 많은 것을 알려준 것에 감사합니다. 이인규 동기는 항상 밝은 모습으로 옆에서 많은 것을 알려주고 도와줬으며 매번 프로젝트에 대한 권유나 좋은 전공 조언을 많이 알려준 것에 감사합니다. 같은 학년을 보내고 모르는 것에 대한 질문도 잘 답해주고 공부하는 데 있어서 도움을 준 김어진 후배에게 감사합니다. 학과 활동 이외에 많은 도움을 주며 긍정적인 모습으로 좋은 영향을 준 정요원 후배에게 감사합니다. 연구실 옆자리에서 항상 밝고 좋은 모습과 함께 힘들 때마다 열심히 응원해준 이은정 후배에게 감사합니다. 이외에 같이 학교생활을 보낸 20학번 동기 친구들 그리고 학과 생활을 하면서 만났던 모든 선후배 모두 감사합니다.

마지막으로 이 논문의 작성을 지도해주시고 연구실 세미나를 통해 좋은 가르침과 도움을 주신 이상정 교수님, 4년 동안 많은 것을 가르쳐 주신 하상호 교수님, 천인국 교수님, 홍인식 교수님, 이해각 교수님, 남윤영 교수님, 조용원 교수님, 김현우 교수님께 감사합니다. 4년 동안 정말 많은 것을 경험하고 배운 것을 잊지 않고 살아가겠습니다. 감사합니다.